



# WTW

*“What’s the Word?”*

**Stop debating. Start deciding.**

---

## Milestone 1 — Proposal & Requirements

CSE 416-03 · Software Engineering · Fall 2026

Team: Mike · Josh · Alan · Razin



*Better  
Decisions  
Together*



# The Problem

Groups waste too much time trying to decide where to go, what to eat, or what to do.

*Good  
Plans  
Together*

## A TYPICAL GROUP CHAT



## It's not a communication failure — it's a structural one.

-  No one wants to commit first — suggesting an idea risks it getting shot down.
-  Group preferences are scattered and usually revealed one at a time.
-  The option space is too large for a group to reason through quickly.
-  Discovery apps help find places, but they don't help groups agree on one.



**Too many  
options**



**Too many  
opinions**



**No  
decision**

## CURRENT TOOLS FALL SHORT

Great for discovery. Not for group decisions.



**Google Maps**

Finds places,  
not consensus.



**Yelp**

Great for reviews,  
not group alignment.



**Beli**

Helps you discover,  
not decide together.



# Who It's For

## PRIMARY USERS



### College students & young adults

Who make casual, spontaneous plans.



### Small groups

Typically 2–8 people.



### Real-life situations

Friends choosing where to eat, roommates looking for something to do, groups planning a night out, or couples deciding where to go.



### Quick, easy planning

People who want a plan without a long group-chat debate.

## INITIAL FOCUS

### Stony Brook University & Friends



Direct participant recruiting



In-person usability testing



Observe how real groups actually use the app



Honest, constructive feedback from our own circles



*Stony Brook is our starting point for testing and learning — but our vision extends beyond college students.*



*WTW does not need thousands of users to be useful — even a few connected friends can already benefit.*



# WTW Turns Debate Into a Decision

WTW helps a user or group move from too many options to one plan everyone is down for.



*A user or friend group picks a broad category — like food, nightlife, activities, or anything — then swipes through a curated set of nearby options.*



Goal:

**< 30 sec**

to start swiping

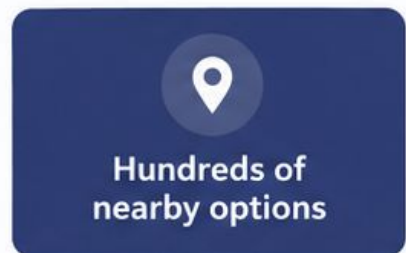


Goal:

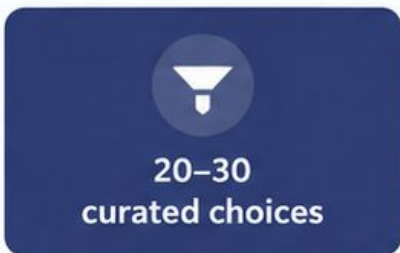
**~ 5 min**

to reach a decision

Instead of endless scrolling and group-chat debate, WTW narrows the search and surfaces where preferences overlap.



Discovery



Filtering



Preference collection



Find the overlap



Decision



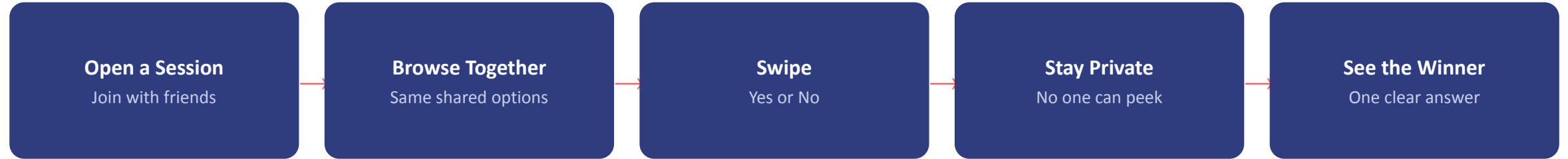
Google Maps and Yelp help you discover places.

***WTW helps you decide together.***

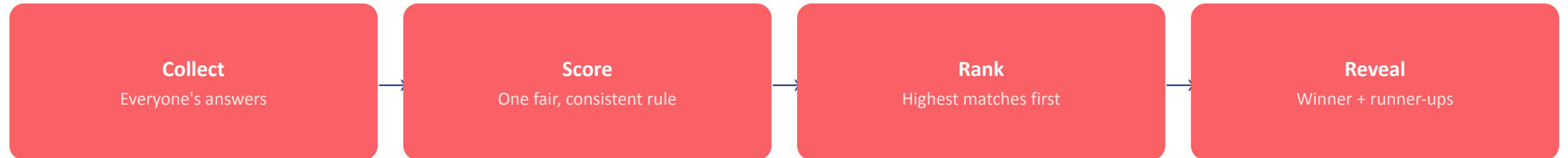


# How WTW Works

## ON THE FRONTEND — WHAT A USER DOES



## BEHIND THE SCENES — HOW IT DECIDES





# Why This Needs a Semester

*This isn't a UI problem. It's several hard engineering problems that only surface once you actually build them.*

1

## Turning Everyone's Location Into One Fair Radius

Participants aren't in the same place — one person's in their dorm, another's already off campus. We have to calculate real distances per person, then decide what “nearby” means for the whole group, not one phone's GPS.

2

## Every Pick Is Its Own Live Session, and There Can Be Many at Once

Different friend groups can be running separate Picks at the same time. Each one needs its own tracked state — who's joined, what's been swiped, when it expires — without sessions leaking into each other.

3

## Keeping the Candidate Pool Fresh, Not Stale

We don't own the place data — it comes from an outside source, and the real world keeps changing under it. Hours shift, places close early, new spots open, old ones shut down for good. If we cache that data to save cost and stay fast, we're choosing to trust information that can go out of date. We need a real strategy for how often to re-scrape and refresh it, or the app ends up sending a group to somewhere that's already closed.

4

## Why This Can't Be Written in One AI-Generated Pass

Location math feeds what candidates show up. Swipe memory changes what the next session looks like. None of these work in isolation, and it all has to hold under real concurrent users — the bugs live in how the pieces interact, and those only surface once we build it, run it, and break it.



# Version 1 Scope

A cross-platform mobile app for iOS and Android, built from one shared codebase

## 1 Accounts & Groups

- Registration, sign-in, profiles, and secure sessions
- Friends and reusable groups
- Create a Pick and invite participants by link

## 2 Place Discovery

- Restaurant results from one external place provider
- Location and distance filters for the group
- Provider adapter so the vendor can be mocked or changed

## 3 Private Group Decision

- Private Yes / Maybe / No responses for each option
- Explainable scoring with a documented tie-break rule
- Top-three vote, winner, and runner-up results

## 4 Reliable Delivery

- Persistent Pick state with live status updates
- Input validation, row-level security, and rate limits
- Automated tests and device testing for the core flow



# Outside Version 1

Intentional cuts protect the semester goal and keep the demo testable

## 1 Additional Platforms

- Separate Swift and Kotlin codebases
- A responsive web or PWA version

## 2 Transactions & Integrations

- Reservations, ticket purchasing, and payments
- Ride-sharing and calendar integrations

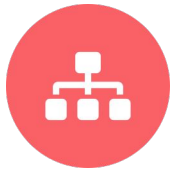
## 3 Advanced Recommendations

- Machine-learning personalization
- Weather, midpoint, and “Adventure Mode” suggestions

## 4 Social & Background Tracking

- Group chat, public feeds, reviews, and dating features
- Automatic history of every location a user visits

*Deferred until the core decision loop works reliably with real groups*



# Architecture & Stack

## BUILD STACK

|          |                            |
|----------|----------------------------|
| APP      | Expo + React Native        |
| LANGUAGE | TypeScript                 |
| DATA     | Postgres + Realtime        |
| IDENTITY | Supabase Auth + RLS        |
| TESTING  | Jest + Maestro             |
| DELIVERY | EAS Build + GitHub Actions |

One mobile codebase targets iOS and Android while managed auth, data, and live sync reduce infrastructure work.

Mobile App — Expo + React Native

Server Boundary — Supabase Edge Functions

Auth + RLS

Postgres  
+ Realtime

Place Provider  
Adapter

External place-data API

Postgres is the source of truth. Realtime carries live Pick updates; Edge Functions protect provider credentials and enforce rate limits.



# Team & Ownership

Each owner leads one system area and supports integration across the full mobile flow



**Alan**

## **Decision Engine & Data Model**

Pick state, private responses, ranking logic, tie-breaking, and result persistence



**Razin**

## **Accounts, Groups & Security**

Authentication, profiles, invitations, authorization, and row-level security



**Josh**

## **Place Provider & Server Services**

Provider adapter, Edge Functions, normalization, caching, rate limits, and mocks



**Mike**

## **Mobile UX & Live Coordination**

Expo navigation, onboarding, swiping, live progress, voting, accessibility, and device testing

# Stop debating. Start deciding.

Questions?



WTW · Mike, Josh, Alan, Razin · CSE 416-03



*Thanks  
for being  
here.*